Velora

VELORA — AI-Powered Visual Regression & Source-Code Investigation Platform

Project Owner: Raghav Bansal

MASTER PROJECT BLUEPRINT | PRD + SOFTWARE DESIGN DOCUMENT

Current working product name: Velora. Preferred by the user, but name availability and trademark/domain clearance have not been verified. The final geometric logo remains to be designed.

Purpose: A professional, resume-worthy, zero-budget-friendly developer platform that compares website versions, identifies visual and responsive regressions, investigates relevant source changes when code is available, and generates evidence-backed AI explanations and reports.

Status: Planning baseline assembled from the web-development assignment guidance and the project decisions made in our conversation. Generated UI mockups are references, not implemented screens or verified scan output.



Executive decisions

Area	Agreed / recommended direction
Product	Visual regression + responsive QA + source-code investigation + optional AI
Modes	URL comparison, screenshot upload, source-aware GitHub/local integration

Frontend	React + TypeScript + Vite; Tailwind CSS + shadcn/ui
Backend	Node.js + Express + TypeScript; Playwright workers
Database	MySQL + Prisma; user accounts and GitHub connections
Design	Electric blue; #080B12 dark / #FFFFFF light; purple rejected
Pages	Seven primary product pages + login/register/reset and settings
Budget	₹0 local-first target; only free AI options; hosting must be evaluated
Brand	Velora is the selected working name; uniqueness not verified

1. Assignment alignment and project scope

Project requirements: select an idea and purpose; define core features and an implementation plan; choose technologies; plan clean logical folders; follow proper Git/GitHub practices; write readable, modular, maintainable code; divide work into milestones; understand how and why the implementation works. Further assignment details may follow.

Problem statement

Frontend updates can accidentally change layout, break responsive behavior, reduce accessibility, or introduce visual inconsistencies. Conventional pixel comparisons show that pixels changed but not necessarily whether the change is harmful or which source edits might be relevant. UI Sentinel connects visual evidence, browser measurements, source diffs, and carefully qualified AI explanations in one investigation workflow.

Goals and non-goals

- Detect and visualize before/after differences across configured viewport sizes.
- Preserve scan history, approved baselines, evidence and downloadable reports.
- Support source-aware investigation for repositories/commits the user authorizes.
- Offer AI explanations, prioritization, suggestions and contextual Q&A grounded in findings.
- Do NOT promise that every pixel change is a bug or that the exact faulty line can always be found.
- Do NOT claim to train a new foundation LLM; integrate existing text/vision models.
- Do NOT automatically execute unknown repository code or apply suggested patches without explicit review.

Personas and example use cases

- Solo React developer compares a local app before/after CSS changes.
- Student demonstrates a deliberately introduced navbar overflow and a verified report.
- Developer connects a GitHub repo and two commits to see changed lines correlated with a detected regression.
- Reviewer exports an evidence-backed report for discussion or evaluation.

2. Functional modes and workflow

Mode	Inputs	Results / limitations
Screenshot Mode	Before/after uploaded screenshots; viewport label	Pixel diff, slider, heatmap, region list; no real DOM/source attribution
Website Mode	Two accessible URLs or approved baseline; viewports	Playwright captures, image diffs, DOM overflow and accessibility checks
Developer Mode	Website evidence + authorized repo/local source + commit refs	Git diff, source tree, possible file/line candidates, AI explanation; confidence labeled

1. Create project or choose existing one.
2. Choose baseline and comparison target; configure viewports, waiting conditions and masking.
3. Run capture jobs and collect screenshots, DOM measurements and accessibility evidence.
4. Compare artifacts; group findings and apply deterministic severity rules.
5. Optionally correlate findings with source changes and request AI interpretation.
6. Review evidence, approve intentional changes, export report and save history.

Localhost rule: a locally running Playwright service can reach local web apps; a remotely deployed server cannot access the user's laptop via its own localhost. Public URL scanning requires strict network/SSRF protections.

3. Seven-page information architecture

Page / route	Core components and functions
1 Dashboard /dashboard	Project overview; real scan metrics; recent activity; trend graph; quick new-scan action; status summaries
2 Projects /projects	Create/edit project; URLs; baseline history; GitHub/local source association; viewport profiles; scan settings
3 New Scan /scans/new	Choose upload, URL or developer mode; before/after targets; viewports; scan options; validation; progress
4 Comparison /scans/:id	Before/after slider; overlay/heatmap; responsive tabs; zoom; issue markers; DOM evidence; finding list
5 Code Investigator /scans/:id/code	Repository tree; Git commit diff; Monaco editor; file/line candidates; evidence links; confidence
6 AI Investigator /scans/:id/ai	Visual analysis; ranked findings; possible root causes; suggestions; issue-scoped chat; provenance
7 Reports /reports	Scan history; filters; issue distribution; trends; detailed report; PDF/JSON export; baseline review

Secondary interfaces: Settings (appearance, AI provider, local paths and privacy), project detail, finding detail and optional authentication. These need not all be separate primary navigation pages.

Navigation behavior

- Desktop: compact collapsible left sidebar, search, project switcher, profile/settings.
- Tablet: narrower navigation with adaptive grids; comparison stays the main content.
- Mobile: accessible drawer/bottom navigation as appropriate, stacked cards and touch-friendly comparison.
- All controls have real actions, loading/error/empty states, keyboard focus, tooltips where useful, and reduced-motion support.

4. Visual system: latest approved direction

Token	Dark mode	Light mode
Main background	#080B12	#FFFFFF
Card / surface	#111827	#F5F8FC
Primary electric blue	#168BFF	#0866D8
Primary text	#F8FAFC	#111827
Secondary text	#94A3B8	#64748B
Borders	Subtle cool charcoal	Pale blue-gray
Semantic states	Green success / amber warning / red critical	Same semantics with accessible contrast

Visual direction: premium, uncluttered developer SaaS; crisp typography; generous whitespace; restrained gradients; limited glow; subtle elevation; strong hierarchy. The electric blue is drawn from the user's visual reference. The earlier purple design is superseded and must not appear in deliverables.

Typography: Inter for interface text; JetBrains Mono for code, diffs, paths and diagnostics. Logo: custom geometric before/after / diagnostic mark, not the generic eye/scan icon. Final artwork remains to be approved. Velora is the current working name, pending uniqueness checks.

5. Motion, transitions and charts

Area	Animation / interaction	Purpose and constraints
Page transitions	150–250ms subtle fade/slide; avoid heavy full-page movement	Fast perceived navigation; reduced-motion fallback
Cards and buttons	Small hover elevation, border emphasis, focus ring, press feedback	Discoverability without distracting glow
Sidebar	Smooth width change with stable content layout	Clear navigation; no jumpy resizing
Scan progress	Stage-based progress: queued, capture, compare, analyze, report	Real state only; no fabricated progress
Comparison slider	60fps pointer drag where possible; synchronized zoom; diff overlay toggle	Central product demonstration
Finding selection	Animate highlight box and scroll into relevant region	Connect evidence to issue
AI responses	Progressive structured sections and status updates	Show actual analysis status, not pretend reasoning
Source explorer	Expand/collapse tree; selected line focus; diff transition	Trace suspected changes
Report graphs	Line chart trend; donut categories; viewport matrix; issue severity bars	All values from persisted scan results
Export	Download state and confirmation/error feedback	No fake downloads

Suggested libraries: Motion for React for UI animation; Recharts for charts; Monaco Editor for code; React Flow only if a dependency/source graph is genuinely added. Prefer CSS transitions for simple effects.

Graph definitions and integrity

- Scan activity trend: count of completed scans over time, with selectable date range.
- Issues over time: counts per scan or approved baseline; explain denominators.
- Category donut: visual/layout/accessibility/source-related findings by category.
- Severity bars: critical/high/medium/low based on defined rules; AI priority shown separately.
- Responsive viewport matrix: findings per viewport width and height.
- Performance measurements only if actually collected; avoid unearned “quality score” or misleading accuracy percentages.

6. Core detection and report engine

- Playwright: capture pages at deterministic viewport sizes, device scale factor, font load and wait strategy.
- sharp + pixelmatch: normalize images and calculate pixel differences; expose threshold and changed-pixel percentage.
- Mask or ignore volatile regions (timestamps, animations, ads, rotating banners) to reduce false positives.
- DOM evidence: element rectangles, computed styles, viewport overflow, visibility and missing/moved selectors where comparable.
- axe-core: automate supported accessibility checks; distinguish tool-reported issues from full WCAG compliance.

- Findings: category, severity, status, viewport, bounding box, before/after evidence, measured values and confidence.
- Baseline approval and intentional-change review are essential because change does not imply regression.

7. AI integration: complete feature specification

AI capability	Evidence input	Output and guardrail
Visual explanation	Before/after crops, diff regions, measurements	Plain-language suspected issue; avoid claiming certainty
CSS/code suggestions	Relevant DOM, styles, source excerpts, Git diff	Suggested patch/snippet; never auto-apply
Prioritization	Deterministic severity + evidence + business impact hints	Ranked queue; distinguish rule score from AI judgment
Report narrative	Verified findings, screenshots, timestamps	Readable executive and technical summaries
Contextual Q&A	Selected issue, scan data, permitted source context	Answers tied to current scan; no generic unsupported claims

Model strategy

Use a provider abstraction with a text-capable model and optionally a vision-capable model. A hosted API can be easier but may cost money or have changing free-tier limits. Ollama/local open-weight models avoid per-call API fees but depend on hardware and model capability. The product must remain useful with AI disabled or unavailable.

AI request pipeline

7. Select relevant finding(s) and evidence, not entire repositories by default.
8. Sanitize context and remove secrets/tokens; obtain consent before sending code/screenshots to a hosted provider.
9. Send a versioned structured prompt with bounded input and requested JSON schema.
10. Validate output schema; label hypotheses and confidence; attach evidence references.
11. Cache/store model/provider/prompt version and response for reproducibility.
12. Allow user review and correction; never silently change code.

8. GitHub, source paths and line attribution

Exact line numbers are only possible where the relevant source and version history are available. A URL-only screenshot cannot reliably reveal original private source files. Developer Mode correlates browser findings with authorized source information; it does not magically prove root cause.

13. Read repository tree and selected commit pair via authorized GitHub integration or approved local folder.
14. Calculate Git diff to obtain actual changed file paths and line ranges.
15. Inspect React/TSX component hierarchy and imports with TypeScript AST / ts-morph.
16. Map affected DOM elements to likely components and styles using selectors, development tooling, and source maps where available.
17. Rank plausible candidate lines and show the evidence linking each candidate to the UI finding.
18. Display actual changed lines separately from suspected cause lines; show “unknown” when attribution is insufficient.

Security: use least-privilege GitHub permissions, encrypt secrets at rest, never commit access tokens, restrict local filesystem access, do not clone/execute arbitrary untrusted code on a public server, and do not promise support for every framework in v1. Prioritize React/TypeScript repositories.

9. Proposed technology architecture

Layer	Technology / responsibility
Client	React + TypeScript + Vite; React Router; Tailwind + shadcn/ui; Motion; Recharts; Monaco
HTTP API	Node.js + Express + TypeScript; validation, project/scan/report endpoints
Background execution	Separate Playwright capture and image-comparison workers; job status/polling
Data	MySQL + Prisma for projects/scans/findings; local file storage for images and report artifacts
Detection	Playwright, sharp, pixelmatch, axe-core, DOM measurement
Source intelligence	GitHub REST API, Git CLI for approved local sources, ts-morph/TypeScript AST
AI	Provider-agnostic service; local Ollama or optional hosted model; schema-validated output
Reporting	HTML report and PDF generation via browser/PDF engine; JSON export
Testing	Vitest/unit tests, API integration tests, Playwright E2E; fixture websites

High-level flow

React UI → Express API → scan record in MySQL → worker captures browser artifacts → comparison/DOM/accessibility findings → optional source investigator → optional AI interpreter → persisted report → interactive dashboard and export.

10. Database model — MySQL

Table	Important fields / relationships
users	id, name, email, password_hash nullable, email_verified_at, preferences; GitHub OAuth identities
projects	id, owner_id nullable, name, target_url, settings_json, created_at
baselines	id, project_id, scan_id, commit_sha, approved_at
scans	id, project_id, baseline_id, status, mode, config_json, started_at, completed_at
scan_artifacts	id, scan_id, kind, viewport, path, metadata_json
findings	id, scan_id, category, severity, confidence, selector, bbox_json, evidence_json, status
source_references	id, finding_id, repo_path, commit_sha, start_line, end_line, attribution_type, confidence
ai_analyses	id, scan_id, finding_id nullable, provider, model, prompt_version, output_json, created_at
ai_messages	id, scan_id, finding_id nullable, role, content,

	created_at
github_connections	user_id, GitHub identity/install reference, permission scopes, encrypted credential reference

Use relational IDs, foreign keys, migrations and indices on scan/project/time. Store large screenshots on disk/object storage, not as giant MySQL blobs. Define retention/cleanup rules for scan artifacts.

11. API surface (initial contract)

Method	Endpoint	Purpose
POST	/api/projects	Create project
GET	/api/projects	List projects
GET/PATCH	/api/projects/:id	Read/update project
POST	/api/scans	Create scan job
GET	/api/scans/:id	Scan status and summary
GET	/api/scans/:id/findings	Finding list/filter
GET	/api/scans/:id/artifacts	Screenshot/diff metadata
POST	/api/scans/:id/ai-analysis	Generate grounded AI explanation
POST	/api/ai/chat	Issue-scoped AI Q&A
GET	/api/scans/:id/source	Source candidates / changed lines
POST	/api/baselines	Approve reference scan
GET	/api/reports/:id/export?format=pdf|json	Download report

All API endpoints require request validation, structured errors, authorization where applicable, and rate/size limits. A public URL scanner must defend against SSRF and browser sandbox escapes; restrict redirects, protocols, hostnames and internal addresses.

12. Repository / folder structure

```
ui-sentinel/
├── apps/
│   ├── web/
│   │   ├── public/
│   │   └── src/
│   │       ├── components/{layout,ui,comparison,findings,charts}/
│   │       ├── pages/{Dashboard,Projects,NewScan,Comparison,CodeInvestigator,AIInvestigator,Reports}.tsx
│   │       ├── hooks/
│   │       ├── services/
│   │       ├── types/
│   │       ├── utils/
│   │       ├── App.tsx
│   │       └── main.tsx
│   └── api/
│       ├── src/
│       │   ├── routes/
│       │   ├── controllers/
│       │   ├── services/{scanner,comparison,accessibility,source-analysis,ai,reports}/
│       │   ├── workers/
│       │   ├── middleware/
│       │   ├── config/
│       │   └── server.ts
│       └── prisma/schema.prisma
├── packages/shared/src/types/
├── tests/{fixtures,integration,e2e}/
├── docs/{requirements,architecture,milestones}.md
├── .env.example
├── .gitignore
├── README.md
└── package.json
```

13. GitHub development practices

- Create a clean repo with README, requirements, architecture notes, setup guide and sample environment file.
- Use feature branches, small scoped commits, descriptive messages and pull requests when helpful.
- Never commit .env, keys, database dumps, scan data with private content, node_modules or generated artifacts.
- Maintain issue/milestone tracking and a changelog or release notes for demonstrations.
- Document commands, dependencies, why each library is used, testing and limitations.

14. Implementation milestones and acceptance tests

Phase	Deliverable / acceptance
M1 Foundation	Responsive dashboard, GitHub OAuth + email/password, secure sessions, projects CRUD, MySQL migration, theme toggle
M2 Comparison MVP	Upload two screenshots, render slider, compute diff, highlight changes, export initial report
M3 Browser automation	Compare URLs/localhost locally; capture desktop/tablet/mobile; overflow + axe findings; scan history
M4 AI investigator	At least one real evidence-grounded model

	explanation; structured output and no-AI fallback
M5 Source-aware investigation	GitHub/local source tree; actual changed lines; cautious candidate attribution; Monaco diff
M6 Polish	Reports/charts, accessible animations, robust errors, tests, security, README and demo

Scheduling: no instructor deadline is confirmed. Build milestone-by-milestone, verify each stage, and do not rush. The full scope may take months part-time. Preserve a working version after each milestone.

15. Risks, mitigations and honest claims

Risk	Mitigation
Dynamic pages cause false positives	Stable capture strategy; mask regions; baseline approval; explicit thresholds
Localhost fails on cloud deployment	Local runner for college demo; clear deployment model
AI invents exact causes	Evidence-linked hypotheses; source availability checks; confidence/unknown state
Source mapping is incomplete	Support React/TS first; show Git diff facts separate from hypotheses
Playwright runs are slow	Background workers, timeouts, retries, scan queue and cleanup
Untrusted URLs / code	SSRF protections, sandboxing, auth, least privilege; no arbitrary execution
Hosted AI costs / downtime	Free-only local Ollama or eligible free hosted tier, strict limits, deterministic fallback
Scope grows beyond deadline	M2/M3 cut line; defer optional integrations and advanced graphs
Brand conflicts	Trademark/domain/GitHub searches before finalizing public name

16. Demo scenario and evaluation story

19. Open a controlled React fixture or own website and save baseline screenshots.
20. Intentionally introduce a responsive regression such as a fixed-width navigation container.
21. Run a new scan at desktop/tablet/mobile viewports.
22. Show the before/after slider and measured overflow at mobile width.
23. Open finding evidence and accessibility results.
24. If source mode is configured, show actual Git changed lines and qualified candidate attribution.
25. Ask AI for a likely explanation; show evidence and suggested fix as a proposal, not a proven answer.
26. Correct the bug, rerun, compare results and export a report.

17. Open decisions (not silently finalized)

- Velora is the current working name; check existing products, GitHub, domain and trademarks before public release. Design a new geometric logo/favicon.
- Exact deadline and any additional instructor rubric are unknown; timeline remains flexible.
- Both GitHub repository and local-folder source analysis are agreed; sequence to be chosen by implementation risk.
- Only no-cost AI options are acceptable; benchmark local Ollama versus genuinely free hosted tiers, and provide no-AI fallback.
- GitHub OAuth and traditional email/password login are required from milestone 1; private repository permission and cloud worker hosting remain later decisions.
- Whether performance audits/SEO/security appear later: not part of guaranteed initial scope unless actually measured.

18. Source of this blueprint

Compiled from the assignment message and the planning/design discussion available in this conversation, including the referenced WhatsApp assignment guidance as described. No additional unseen WhatsApp messages have been independently inspected. All illustrative numbers and screenshots in generated mockups are concept art, not measured product results.

19. Visual references from our design iterations

LATEST BLUE DIRECTION — primary visual reference. Layouts are conceptual and should be cleaned up further when implementing individual pages.



Original Pinterest screenshot: inspiration for electric-blue accent and premium atmosphere; do not copy another brand's imagery or design wholesale.

20. Confirmed decision register — 29 October 2026

Topic	Confirmed direction / caveat
Name	Velora working name; not verified as unused
Theme	Electric blue on near-black and white; NO purple visual references
Logo	New original geometric logo; artwork pending
Login	GitHub OAuth primary + email/password registration/login + password reset

Google login	Not planned for initial release
Source access	Both public/authorized GitHub repositories and local folders
AI	All five features on roadmap; only zero-cost options; provider TBD after testing
Deployment	Vercel preferred for frontend; GitHub Pages alternative for static UI; backend, MySQL and Playwright require separate hosting
Time	No fixed deadline confirmed; quality-first milestones
Development	Build, test and finish each milestone before moving to next
Integrity	No fabricated metrics, reports, AI certainty or fake controls

21. Authentication and deployment specification

- Landing/sign-in screen: primary Continue with GitHub; secondary email/password; register, forgot password, reset password, sign out and clear error messages.
- Use a maintained authentication library and secure session strategy; hash passwords, protect CSRF where applicable, use secure cookies, validation, email verification and rate limiting.
- GitHub identity login is not equivalent to permission to access repositories; request least-privilege repository authorization separately.
- Vercel can host the frontend; GitHub Pages can host a static SPA with routing accommodations. Neither automatically hosts persistent Express/Playwright workers or MySQL.
- Start with local MySQL, local backend and local Playwright. Decide hosted worker/database provider only after measuring resource needs and zero-cost limits.
- Keep secrets in environment variables, provide .env.example, and document OAuth redirect URLs for local and deployed environments.

22. Free AI policy and evaluation

- No paid model subscriptions or mandatory API spending. Provider abstraction supports local Ollama and any eligible genuinely free hosted API, with limits disclosed.
- Benchmark vision support for screenshot analysis, text reasoning for DOM/source evidence, structured JSON validity, latency, local GPU/RAM needs and rate limits.
- If no suitable free vision model is available, retain screenshot and deterministic findings, and feed text evidence to a text model; do not pretend the model inspected images.
- All five AI features remain planned: visual explanations, source-aware CSS suggestions, issue ranking, report narratives and contextual issue chat.
- AI claims must link to screenshot/DOM/Git evidence; differentiate confirmed changed lines from suspected root-cause candidates.
- Never auto-apply AI code edits. Show proposals and require developer review.

23. Complete start-to-finish implementation checklist

Phase 0 — Requirements and preparation

☐ Confirm instructor rubric/deadline when available; keep current flexible scope.

☐ Check Velora name collisions, domains, GitHub organization/repository availability and trademarks; rename if necessary.

☐ Finalize geometric logo, favicon, dark/light palette, typography, spacing and component tokens.

☐ Draw wireframes for login, dashboard, projects, new scan, comparison, code, AI and reports.

☐ Create GitHub repository, branch policy, issues/project board, README, license decision and .gitignore.

☐ Document user journeys, privacy/security boundaries, MVP acceptance criteria and out-of-scope items.

Phase 1 — Workspace, design system and authentication

☐ Initialize monorepo with React/Vite/TypeScript frontend, Express/TypeScript backend and shared types.

☐ Set up Tailwind/shadcn UI, routing, form validation, lint/format/test scripts and environment templates.

☐ Implement black/blue and white/blue themes, persistence, accessible UI controls and responsive sidebar.

☐ Design/build login, registration, reset password, GitHub sign-in and authentication callback states.

☐ Implement secure user/session persistence, MySQL + Prisma schema and migrations.

☐ Create authenticated route guards, logout, rate limits, loading/error states and auth tests.

☐ Build real project CRUD, project settings and initial dashboard with actual database records.

☐ Complete and review milestone 1 before starting screenshot comparison.

Phase 2 — Screenshot comparison MVP

☐ Create screenshot upload UI with file validation, size/type limits and clear errors.

☐ Implement secure artifact storage and metadata records; prevent unsafe file paths.

☐ Implement sharp/pixelmatch diff, configurable thresholds and region grouping.

☐ Build synchronized before/after slider, overlay, heatmap, zoom and issue markers.

☐ Create comparison summary and initial downloadable HTML/JSON report.

☐ Test identical, shifted, resized and intentionally changed image fixtures.

☐ Add empty/loading/error states, cleanup and regression tests.

Phase 3 — Browser automation and responsive diagnostics

☐ Implement queued scan job model, worker process, polling and cancellation/timeouts.

☐ Build URL input validation, localhost scanning in local environment and safe browser contexts.

☐ Capture consistent screenshots at desktop, tablet and mobile viewports.

☐ Control animation timing, fonts, image loading, dynamic content and masks for stable baselines.

☐ Collect DOM element geometry, viewport overflow and basic layout measurements.

☐ Integrate axe-core accessibility findings and link them to screenshots where possible.

☐ Implement baseline approval, scan history, rerun and multi-viewport comparison.

☐ Test SSRF protections, redirect handling, resource limits and browser cleanup.

Phase 4 — AI investigator (free-only)

☐ Build AI provider interface and choose/test genuinely free model options.

☐ Create evidence bundle format: screenshots when supported, diff regions, DOM findings, viewport and source references.

☐ Implement structured prompts and validated JSON responses with uncertainty markers.

☐ Add visual finding explanation and severity/prioritization with deterministic fallback.

☐ Add contextual issue-specific AI Q&A and source-aware CSS suggestions when source exists.

☐ Generate grounded report narratives and persist model/provider metadata.

☐ Add rate-limit handling, timeouts, privacy controls and prompt-injection defenses.

☐ Evaluate AI on known fixture bugs and log false positives/unsupported claims.

Phase 5 — GitHub and local source intelligence

☐ Implement public/authorized GitHub repository selection and least-privilege permissions.

☐ Implement local project folder selection in a safe local agent/runner model; do not assume a cloud browser can read laptop files.

☐ Build file tree explorer, Git commit picker, changed file and changed-line views.

☐ Implement React/TS component/import analysis and CSS selector/source-map investigation where available.

☐ Correlate DOM/selector evidence with candidate files and changed lines.

☐ Display confirmed Git diff facts separately from AI-suspected root causes.

☐ Integrate Monaco code diff and evidence-linked navigation from findings to source.

☐ Test monorepos, absent source maps, Tailwind, CSS modules and unresolvable cases.

Phase 6 — Complete page designs and reporting

☐ Polish all seven primary pages and secondary settings/auth flows in both themes.

☐ Implement meaningful Motion transitions, accessible reduced-motion fallback and responsive micro-interactions.

☐ Add Recharts regression trends, issue-category distribution, severity breakdown and viewport matrix.

☐ Build report detail with screenshots, evidence, code candidates, AI explanations and scan metadata.

☐ Implement PDF, HTML and JSON export, verifying charts/print styles and file correctness.

☐ Add filtering, sorting, saved scan history, baseline comparison and project overview charts.

☐ Ensure every button and chart is wired to actual data or clearly marked unavailable.

Phase 7 — Quality, security and deployment

☐ Run unit, integration and E2E tests; add reproducible broken-site fixtures.

☐ Audit keyboard navigation, contrast, focus, screen-reader labels, mobile layouts and reduced motion.

☐ Review auth/session security, repository permissions, upload validation, SSRF and secret handling.

☐ Measure screenshot stability, job timeouts, memory use and model latency; tune queue/resource limits.

☐ Deploy frontend to Vercel or GitHub Pages; choose a viable separate backend/worker/MySQL solution.

☐ Configure production environment variables, HTTPS, OAuth callback URLs, logs and monitoring.

☐ Test deployed login, projects, scanning, AI fallback, source investigation and report exports.

☐ Write README, architecture diagrams, setup commands, limitations, demo guide and screenshots.

☐ Record real demo: baseline → deliberate UI regression → detection → source candidate → fix → rescan → report.

☐ Tag release, verify GitHub repo hygiene, prepare evaluation speech and portfolio/resume summary.

24. Milestone completion rule

Each milestone is done only when its real user journey works end-to-end, validation and failure states are implemented, tests pass, code is committed, and README/docs are updated. Never replace unfinished features with misleading demo data.

25. Approved visual references — BLUE ONLY

Electric-blue concept image is the sole included design reference. The older purple mockup and Nexora concept have been removed from this document. The blue concept remains a moodboard, not a finished specification for every screen.



Document end — v2.0, consolidated decisions and end-to-end task register.